

```
In [1]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import sqlite3
import warnings
warnings.filterwarnings("ignore")
```

```
In [2]: #1. import database/data
```

```
In [3]: conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table'

"""

tables = pd.read_sql(sql_query, conn)

for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

Created dataframe: df\_db\_customer

Created dataframe: df\_db\_subscription

Created dataframe: df\_db\_support

```
In [4]: conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")

    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

conn.close()
```

Table Name: db\_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db\_subscription

Columns:

['customerid', 'subscription\_start\_date', 'subscription\_type', 'renewal\_date', 'plan\_type', 'contract\_type', 'cancellation\_date', 'cancellation\_reason', 'monthly\_charges', 'cltv', 'churn\_score']

Table Name: db\_support

Columns:

['customerid', 'complaint\_date', 'escalations', 'csat\_score', 'col\_1', 'comment']

In [5]: `# 2. data cleaning`

In [6]: `#df_db_customer.head()  
df_db_customer.tail()`

Out[6]:

|    | customerid | name       | country | state         | gender | dob                 | interests | pincode |
|----|------------|------------|---------|---------------|--------|---------------------|-----------|---------|
| 16 | 0020-JDNP  | rikim      | India   | Meghalaya     | Female | 1994-08-19 00:00:00 | None      | None    |
| 17 | 0021-IKXGC | vishakha   | India   | Rajasthan     | Female | 2000-09-02 00:00:00 | None      | None    |
| 18 | 0022-TCJCI | raghvendra | India   | Telangana     | Male   | 1983-12-30 00:00:00 | None      | None    |
| 19 | 0023-HGHWL | rishabh    | India   | Uttar Pradesh | Men    | 1991-05-14 00:00:00 | None      | None    |
| 20 | 0023-UYUPN | sudevi     | India   | Maharashtra   | Women  | 1977-10-06 00:00:00 | None      | None    |

In [7]: `df_db_customer.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    object
1   name        21 non-null    object
2   country     18 non-null    object
3   state       21 non-null    object
4   gender      21 non-null    object
5   dob         21 non-null    object
6   interests   4 non-null     object
7   pincode     0 non-null     object
dtypes: object(8)
memory usage: 1.4+ KB
```

```
In [8]: # a.rename col name  
# b.drop column - interest and pincode  
# c.change data type - dob  
# d. data standardization - gender  
# e. fix missing values - country
```

```
In [9]: # a.rename col - name  
  
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)  
df_db_customer
```

Out[9]:

|    | customerid | customer_name | country | state         | gender | dob                    | interests | pincode |
|----|------------|---------------|---------|---------------|--------|------------------------|-----------|---------|
| 0  | 0002-ORFBO | keshav        | India   | Maharashtra   | Male   | 1982-04-12<br>00:00:00 | travel    | N       |
| 1  | 0003-MKNFE | raghav        | India   | Karnataka     | Male   | 1995-11-23<br>00:00:00 | None      | N       |
| 2  | 0004-TLHLJ | lalita        | India   | Delhi         | Female | 1978-02-15<br>00:00:00 | movie     | N       |
| 3  | 0011-IGKFF | mohan         | India   | Nagaland      | Male   | 2001-08-30<br>00:00:00 | None      | N       |
| 4  | 0013-EXCHZ | mira          | India   | Delhi         | Female | 1990-05-05<br>00:00:00 | drama     | N       |
| 5  | 0013-MHZWF | durga         | None    | Delhi         | Women  | 1988-12-10<br>00:00:00 | None      | N       |
| 6  | 0013-SMEOE | mina          | India   | Meghalaya     | Female | 1976-09-21<br>00:00:00 | None      | N       |
| 7  | 0014-BMAQU | madan         | India   | Rajasthan     | Male   | 1999-03-14<br>00:00:00 | None      | N       |
| 8  | 0015-UOCOJ | maya          | None    | Kathmandu     | Women  | 1985-07-07<br>00:00:00 | None      | N       |
| 9  | 0016-QLJIS | arjun         | Nepal   | Kathmandu     | Male   | 1993-10-29<br>00:00:00 | None      | N       |
| 10 | 0017-DINOC | shiva         | India   | Maharashtra   | Men    | 1997-01-22<br>00:00:00 | None      | N       |
| 11 | 0017-IUDMW | rangadevi     | India   | Karnataka     | Female | 1981-06-18<br>00:00:00 | None      | N       |
| 12 | 0018-NYROU | chitra        | None    | Telangana     | Female | 2004-12-01<br>00:00:00 | None      | N       |
| 13 | 0019-EFAEP | raju          | India   | Meghalaya     | Female | 1992-04-25<br>00:00:00 | None      | N       |
| 14 | 0019-GFNTW | Madhav        | India   | Uttar Pradesh | Men    | 1979-11-11<br>00:00:00 | None      | N       |

|    | customerid | customer_name | country | state         | gender | dob                 | interests | pincode |
|----|------------|---------------|---------|---------------|--------|---------------------|-----------|---------|
| 15 | 0020-INWCK | parvati       | India   | Delhi         | Female | 1986-02-28 00:00:00 | job       | N       |
| 16 | 0020-JDNPX | rikim         | India   | Meghalaya     | Female | 1994-08-19 00:00:00 | None      | N       |
| 17 | 0021-IKXGC | vishakha      | India   | Rajasthan     | Female | 2000-09-02 00:00:00 | None      | N       |
| 18 | 0022-TJCI  | raghvendra    | India   | Telangana     | Male   | 1983-12-30 00:00:00 | None      | N       |
| 19 | 0023-HGHWL | rishabh       | India   | Uttar Pradesh | Men    | 1991-05-14 00:00:00 | None      | N       |
| 20 | 0023-UYUPN | sudevi        | India   | Maharashtra   | Women  | 1977-10-06 00:00:00 | None      | N       |

In [10]: # b. drop column - interest and pincode

```
#df_db_customer.drop(df_db_customer.columns[-2:], axis=1)

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)
```

In [11]: # c. change data type - dob

```
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [12]: # d. data standardization - gender

```
#df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Wo
```

In [13]: df\_db\_customer['gender'].unique()

Out[13]: array(['Male', 'Female'], dtype=object)

In [14]: # e. fix missing values - country

```
df_db_customer[df_db_customer['country'].isna()]
```

Out[14]:

|    | customerid | customer_name | country | state     | gender | dob        |
|----|------------|---------------|---------|-----------|--------|------------|
| 5  | 0013-MHZWF | durga         | None    | Delhi     | Female | 1988-12-10 |
| 8  | 0015-UOCOJ | maya          | None    | Kathmandu | Female | 1985-07-07 |
| 12 | 0018-NYROU | chitra        | None    | Telangana | Female | 2004-12-01 |

```
In [15]: # country and state unique value pair

state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state', 'country')
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping['country']))
```

```
In [16]: df_db_customer[df_db_customer['country'].isna()]
```

```
Out[16]:
```

| customerid | customer_name | country | state | gender | dob |
|------------|---------------|---------|-------|--------|-----|
|------------|---------------|---------|-------|--------|-----|

```
In [ ]:
```

```
In [17]: df_db_subscription.head()
```

```
Out[17]:
```

|  | customerid | subscription_start_date | subscription_type | renewal_date | plan_type | cont |
|--|------------|-------------------------|-------------------|--------------|-----------|------|
|--|------------|-------------------------|-------------------|--------------|-----------|------|

|   |            |            |          |            |          |  |
|---|------------|------------|----------|------------|----------|--|
| 0 | 0002-ORFBO | 2021-03-15 | Refferal | 2025-03-15 | Standard |  |
|---|------------|------------|----------|------------|----------|--|

|   |            |            |      |            |         |  |
|---|------------|------------|------|------------|---------|--|
| 1 | 0003-MKNFE | 2020-08-01 | Paid | 2024-08-01 | Premium |  |
|---|------------|------------|------|------------|---------|--|

|   |            |            |         |            |       |  |
|---|------------|------------|---------|------------|-------|--|
| 2 | 0004-TLHLJ | 2022-11-20 | Organic | 2025-11-20 | Basic |  |
|---|------------|------------|---------|------------|-------|--|

|   |            |            |      |            |         |  |
|---|------------|------------|------|------------|---------|--|
| 3 | 0011-IGKFF | 2019-05-10 | Paid | 2025-05-10 | Premium |  |
|---|------------|------------|------|------------|---------|--|

|   |            |            |          |            |          |  |
|---|------------|------------|----------|------------|----------|--|
| 4 | 0013-EXCHZ | 2023-01-05 | Refferal | 2024-01-05 | Standard |  |
|---|------------|------------|----------|------------|----------|--|



```
In [18]: df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                            21 non-null    object
1   subscription_start_date                21 non-null    object
2   subscription_type                      21 non-null    object
3   renewal_date                          21 non-null    object
4   plan_type                             21 non-null    object
5   contract_type                         21 non-null    object
6   cancellation_date                     6 non-null     object
7   cancellation_reason                   6 non-null     object
8   monthly_charges                       21 non-null    float64
9   cltv                                  21 non-null    int64
10  churn_score                           21 non-null    int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

```
In [19]: date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']
```

```
df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info
```

```

Out[19]: <bound method DataFrame.info of      customerid subscription_start_date subscrip
tion_type renewal_date \
0  0002-ORFBO      2021-03-15      Refferal      2025-03-15
1  0003-MKNFE      2020-08-01      Paid      2024-08-01
2  0004-TLHLJ      2022-11-20      Organic      2025-11-20
3  0011-IGKFF      2019-05-10      Paid      2025-05-10
4  0013-EXCHZ      2023-01-05      Refferal      2024-01-05
5  0013-MHZWF      2022-06-18      Paid      2025-06-18
6  0013-SMEOE      2021-09-30      Refferal      2024-09-30
7  0014-BMAQU      2020-02-14      Organic      2025-02-14
8  0015-UOCOJ      2023-07-22      Organic      2024-07-22
9  0016-QLJIS      2022-04-03      Organic      2025-04-03
10 0017-DINOC      2021-12-01      Organic      2025-12-01
11 0017-IUDMW      2023-03-17      Refferal      2024-03-17
12 0018-NYROU      2020-10-09      Organic      2025-10-09
13 0019-EFAEP      2022-08-25      Refferal      2024-08-25
14 0019-GFNTW      2019-11-11      Paid      2025-11-11
15 0020-INWCK      2023-05-06      Organic      2025-05-06
16 0020-JDNXP      2022-01-19      Organic      2024-01-19
17 0021-IKXGC      2021-07-07      Paid      2025-07-07
18 0022-TCJCI      2023-09-14      Refferal      2024-09-14
19 0023-HGHWL      2020-06-23      Organic      2025-06-23
20 0023-UYUPN      2022-12-31      Paid      2025-12-31

      plan_type contract_type cancellation_date      cancellation_reason \
0  Standard      Annual      NaT      None
1  Premium      Annual      2024-09-10  Switched to competitor
2  Basic      Monthly      NaT      None
3  Premium      Annual      NaT      None
4  Standard      Monthly      2024-02-28      Too expensive
5  Standard      Annual      NaT      None
6  Basic      Monthly      2024-11-15      Not enough content
7  Premium      Annual      NaT      None
8  Standard      Monthly      NaT      None
9  Basic      Annual      NaT      None
10 Premium      Annual      NaT      None
11 Standard      Monthly      2024-05-01  Poor streaming quality
12 Standard      Annual      NaT      None
13 Basic      Monthly      2024-10-31  Switched to competitor
14 Premium      Annual      NaT      None
15 Standard      Monthly      NaT      None
16 Premium      Annual      NaT      None
17 Standard      Annual      NaT      None
18 Basic      Monthly      2024-09-14  Forgot to cancel trial
19 Premium      Annual      NaT      None
20 Standard      Monthly      NaT      None

      monthly_charges  cltv  churn_score
0          13.99    627          12
1          12.99   1150          91
2           6.99    210          34
3          22.99   1725           8
4          13.99    195          88
5          17.99    720          22
6           8.99    230          79
7          22.99   1840           5
8          13.99    240          34
9           6.99    335          41
10         22.99   1610          14
11          7.99    270          83

```

|    |       |      |    |
|----|-------|------|----|
| 12 | 13.99 | 980  | 19 |
| 13 | 12.99 | 160  | 76 |
| 14 | 92.99 | 2185 | 3  |
| 15 | 13.99 | 640  | 58 |
| 16 | 20.99 | 550  | 62 |
| 17 | 13.99 | 840  | 27 |
| 18 | 16.99 | 42   | 99 |
| 19 | 22.99 | 1955 | 7  |
| 20 | 13.99 | 790  | 47 |

In [ ]:

In [20]: df\_db\_support.head()

Out[20]:

|   | customerid | complaint_date      | escalations | csat_score | col_1 | comment           |
|---|------------|---------------------|-------------|------------|-------|-------------------|
| 0 | 0003-MKNFE | 2024-08-28 00:00:00 | N           | 60         | None  | service issue     |
| 1 | 0003-MKNFE | 2024-08-28 00:00:00 | Y           | 10         | None  | demaned refund    |
| 2 | 0013-EXCHZ | 2024-01-20 00:00:00 | Y           | 20         | None  | None              |
| 3 | 0013-MHZWF | 2025-03-18 00:00:00 | N           | 90         | None  | guidance to renew |
| 4 | 0013-SMEOE | 2024-11-01 00:00:00 | N           | 30         | None  | None              |

In [21]: df\_db\_support.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     object
1   complaint_date  9 non-null     object
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     object
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

In [22]: df\_db\_support.drop(columns=['col\_1', 'comment'], inplace=True)

In [23]: df\_db\_support['complaint\_date'] = pd.to\_datetime(df\_db\_support['complaint\_date'])  
df\_db\_support.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            9 non-null     object
1   complaint_date        9 non-null     datetime64[ns]
2   escalations           9 non-null     object
3   csat_score            9 non-null     int64
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 420.0+ bytes
```

In [ ]:

In [24]: *# 3. Feature Engineering and Data Analysis*In [25]: *# create a new column using existing col - churn flag*  
df\_db\_subscription['churn\_flag'] = np.where(df\_db\_subscription['cancellation\_dat

In [26]: df\_db\_subscription.head()

Out[26]:

|   | customerid | subscription_start_date | subscription_type | renewal_date | plan_type | cont |
|---|------------|-------------------------|-------------------|--------------|-----------|------|
| 0 | 0002-ORFBO | 2021-03-15              | Refferal          | 2025-03-15   | Standard  |      |
| 1 | 0003-MKNFE | 2020-08-01              | Paid              | 2024-08-01   | Premium   |      |
| 2 | 0004-TLHLJ | 2022-11-20              | Organic           | 2025-11-20   | Basic     |      |
| 3 | 0011-IGKFF | 2019-05-10              | Paid              | 2025-05-10   | Premium   |      |
| 4 | 0013-EXCHZ | 2023-01-05              | Refferal          | 2024-01-05   | Standard  |      |

In [27]: *# first fix support table duplicates then merge*  
df = (df\_db\_subscription  
      .merge(df\_db\_customer, on = 'customerid', how= 'left' )  
      .merge(df\_db\_support, on = 'customerid', how= 'left' ))

In [28]: df.shape

Out[28]: (23, 20)

In [29]: df\_db\_subscription.shape

Out[29]: (21, 12)

In [30]: df\_db\_customer['customerid'].nunique()

Out[30]: 21

In [31]: df\_db\_subscription['customerid'].nunique()

Out[31]: 21

```
In [32]: df_db_support['customerid'].nunique()
```

```
Out[32]: 7
```

```
In [33]: df_db_support['customerid'].size
```

```
Out[33]: 9
```

```
In [34]: df_db_support
```

```
Out[34]:
```

|   | customerid | complaint_date | escalations | csat_score |
|---|------------|----------------|-------------|------------|
| 0 | 0003-MKNFE | 2024-08-28     | N           | 60         |
| 1 | 0003-MKNFE | 2024-08-28     | Y           | 10         |
| 2 | 0013-EXCHZ | 2024-01-20     | Y           | 20         |
| 3 | 0013-MHZWF | 2025-03-18     | N           | 90         |
| 4 | 0013-SMEOE | 2024-11-01     | N           | 30         |
| 5 | 0017-IUDMW | 2024-04-10     | Y           | 25         |
| 6 | 0019-EFAEP | 2024-09-27     | Y           | 30         |
| 7 | 0022-TCJCI | 2024-09-13     | Y           | 10         |
| 8 | 0022-TCJCI | 2024-09-14     | N           | 90         |

```
In [35]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customer
```

```
In [36]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus
```

```
In [37]: df_db_support['customerid'].size
```

```
Out[37]: 7
```

```
In [38]: # merge df
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left' )
      .merge(df_db_support, on = 'customerid', how= 'left' ))
```

```
In [39]: df.shape
```

```
Out[39]: (21, 21)
```

```
In [40]: df.to_csv('exported_churn_data.csv', index= False)
```

```
In [ ]:
```

```
In [41]: # Data Analysis
```

```
In [42]: # 1. Churn Rate
```

```
In [43]: df.columns
```

```
Out[43]: Index(['customerid', 'subscription_start_date', 'subscription_type',
              'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
              'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
              'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
              'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
              dtype='object')
```

```
In [44]: churn_rate = df['churn_flag'].mean()*100
         print("Churn Rate = ", round(churn_rate,2), "%")
```

Churn Rate = 28.57 %

```
In [45]: # 2. Retention Rate
         retention_rate = 100 - churn_rate
         print("Retention Rate = ", round(retention_rate,2), "%")
```

Retention Rate = 71.43 %

```
In [46]: df.head(2)
```

```
Out[46]:
```

|   | customerid | subscription_start_date | subscription_type | renewal_date | plan_type | cont |
|---|------------|-------------------------|-------------------|--------------|-----------|------|
| 0 | 0002-ORFBO | 2021-03-15              | Refferal          | 2025-03-15   | Standard  |      |
| 1 | 0003-MKNFE | 2020-08-01              | Paid              | 2024-08-01   | Premium   |      |

2 rows × 21 columns



```
In [47]: # 3. Churn by Plan type
         churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).
                           print(churn_by_plan)
```

|   | plan_type | churn_rate_pct |
|---|-----------|----------------|
| 0 | Basic     | 60.00          |
| 1 | Premium   | 14.29          |
| 2 | Standard  | 22.22          |

```
In [48]: # 4 a. Churn by state + sum (revenue) & count of users
         # 4 b. Churn by subscription type + sum (revenue) & count of users
```

```
In [49]: # 5. ARPU - Average Revenue Per User
         arpu = df['monthly_charges'].mean()
         print('ARPU=', round(arpu,2))
```

ARPU= 18.85

```
In [50]: # 6. Average customer tenure
         # count of days usres has used our service : cancellation date else current date
         today = pd.Timestamp.today()

         df['tenure_days'] = np.where(
             df['cancellation_date'].notna(),

             (df['cancellation_date'] - df['subscription_start_date']).dt.days,

             (today - df['subscription_start_date']).dt.days
```

```
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days)", round(avg_tenure),0)
```

Avg Tenure (Days) 1530 0

```
In [51]: # 7. Revenue at risk - revenue llost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1,'monthly_charges'].sum()
print("Revenue at Risk (Rs 'k') =",revenue_at_risk)
```

Revenue at Risk (Rs 'k') = 73.94

```
In [52]: # 8. Esculation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate =", round(escalation_rate, 2), "%")
```

Escalation Rate = 19.05 %

```
In [53]: # 9. Avg complaint per user
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Complaints Per User = ", round(avg_complaints, 2))
```

Avg Complaints Per User = 0.43

```
In [54]: # 10. Correlation Escalation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) #Encoding string
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation, 2))
```

Correlation between escalation vs churn is = 0.77

```
In [55]: # 11. Create a column using existing col - Churn risk
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unknown')
```

In [ ]:

```
In [56]: # 4. Visualizing using Matplotlib
```

```
In [57]: df_visual = df.copy()
```

```
In [58]: df.shape
```

```
Out[58]: (21, 23)
```

```
In [59]: df.columns
```

```
Out[59]: Index(['customerid', 'subscription_start_date', 'subscription_type',
              'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
              'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
              'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
              'complaint_date', 'escalations', 'csat_score', 'complaint_count',
              'tenure_days', 'churn_risk'],
              dtype='object')
```

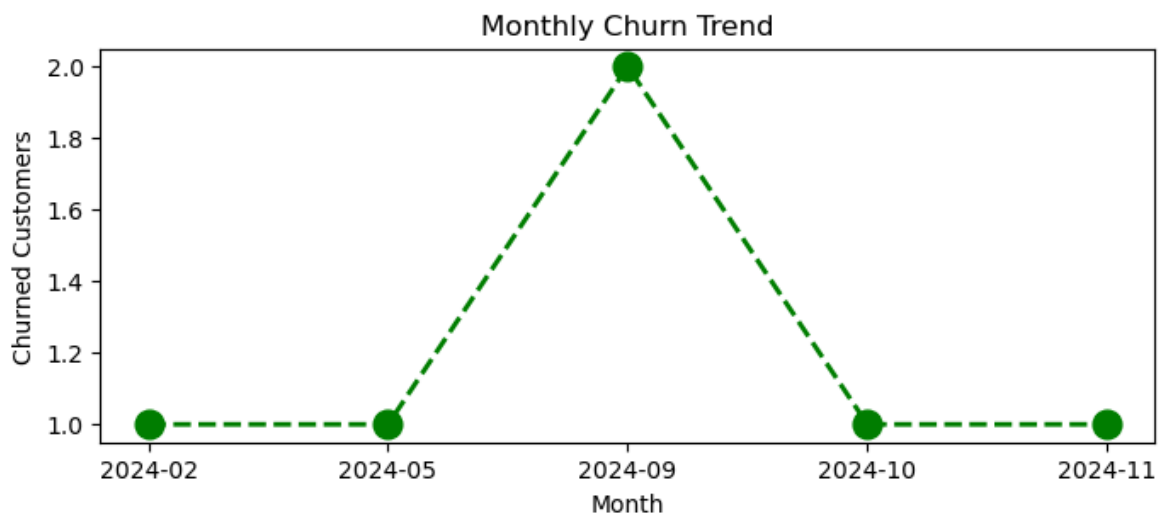
```
In [60]: # 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker='o')
plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show
```

```
Out[60]: <function matplotlib.pyplot.show(close=None, block=None)>
```



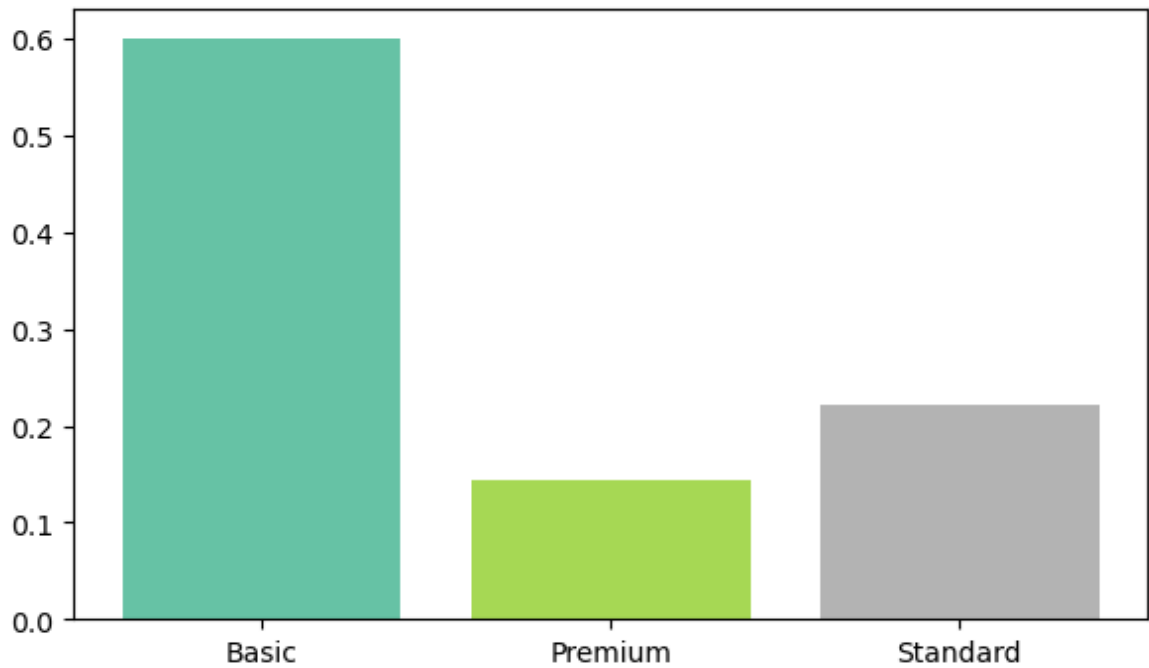
```
In [61]: # 4.2 Churn By Plan Type

churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```

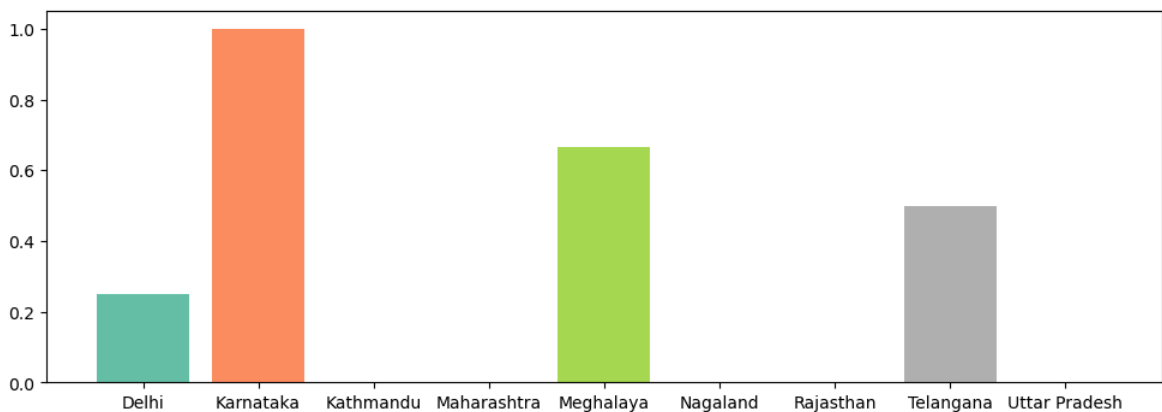


```
In [62]: # 4.3 Churn by State
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```



```
In [ ]:
```

```
In [63]: # Visualization using Seaborn
```

```
In [64]: # encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

```
Out[64]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count',
               'tenure_days', 'churn_risk', 'cancellation_month'],
              dtype='object')
```

In [65]: `df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]`

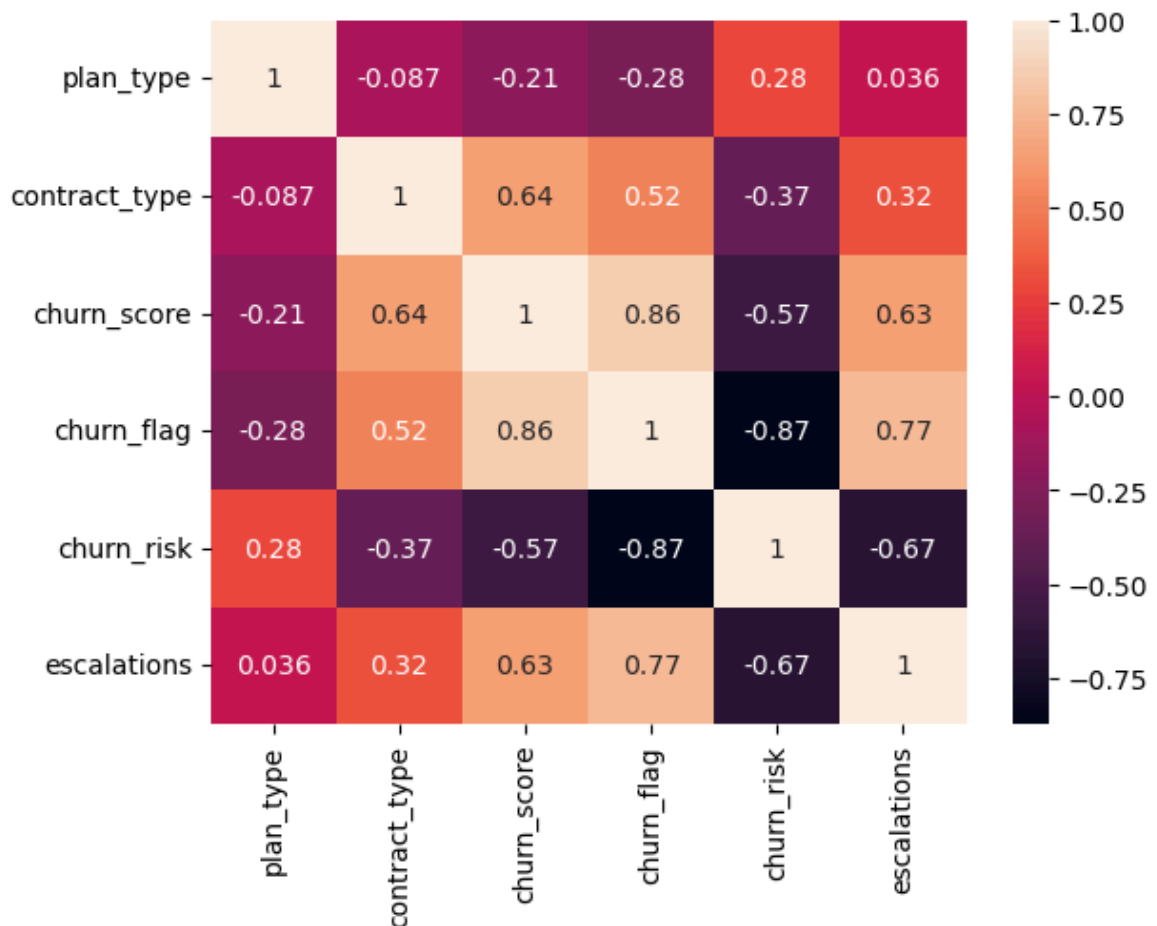
Out[65]:

|   | plan_type | contract_type | churn_score | churn_flag | churn_risk | escalations |
|---|-----------|---------------|-------------|------------|------------|-------------|
| 0 | Standard  | Annual        | 12          | 0          | low        | 0           |
| 1 | Premium   | Annual        | 91          | 1          | high       | 1           |
| 2 | Basic     | Monthly       | 34          | 0          | low        | 0           |
| 3 | Premium   | Annual        | 8           | 0          | low        | 0           |
| 4 | Standard  | Monthly       | 88          | 1          | high       | 1           |

In [67]: `# incorrect method of encoding - as numbers are not assigned based on priority`  
`df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]`  
`categorical_cols = ['plan_type', 'contract_type', 'churn_risk']`  
`for col in categorical_cols:`  
`df_encoded[col] = df_encoded[col].astype('category').cat.codes`

In [68]: `# Heatmap (correlation matrix)`  
`sns.heatmap(df_encoded.corr(), annot=True)`

Out[68]: <Axes: >



In [69]: `df_visual['churn_risk'].unique()`

Out[69]: array(['low', 'high', 'med'], dtype=object)

```
In [70]: # correct method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',
                        'churn_risk']]

order_mappings = {
    'plan_type' : ['Basic', 'Premium', 'Standard'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk' : ['low', 'med', 'high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order)
```

In [71]: df\_visual[['plan\_type', 'contract\_type', 'churn\_score', 'churn\_flag', 'churn\_risk', 'churn\_esc']]

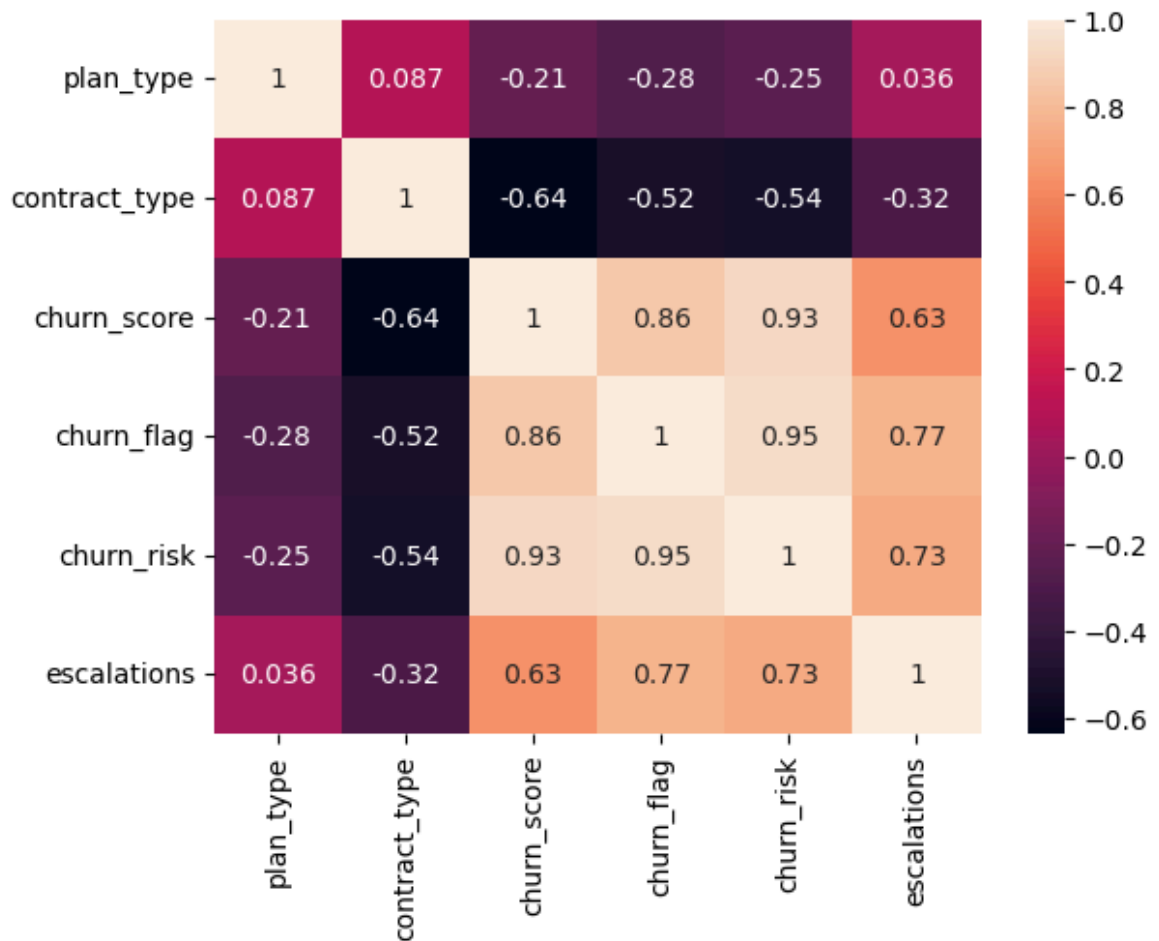
```
Out[71]: <bound method NDFrame.head of DataFrame>
ag churn_risk escalations plan_type contract_type churn_score churn_flag
0 Standard Annual 12 0 low 0
1 Premium Annual 91 1 high 1
2 Basic Monthly 34 0 low 0
3 Premium Annual 8 0 low 0
4 Standard Monthly 88 1 high 1
5 Standard Annual 22 0 low 0
6 Basic Monthly 79 1 high 0
7 Premium Annual 5 0 low 0
8 Standard Monthly 34 0 low 0
9 Basic Annual 41 0 low 0
10 Premium Annual 14 0 low 0
11 Standard Monthly 83 1 high 1
12 Standard Annual 19 0 low 0
13 Basic Monthly 76 1 high 1
14 Premium Annual 3 0 low 0
15 Standard Monthly 58 0 med 0
16 Premium Annual 62 0 med 0
17 Standard Annual 27 0 low 0
18 Basic Monthly 99 1 high 0
19 Premium Annual 7 0 low 0
20 Standard Monthly 47 0 low 0>
```

```
In [72]: # Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

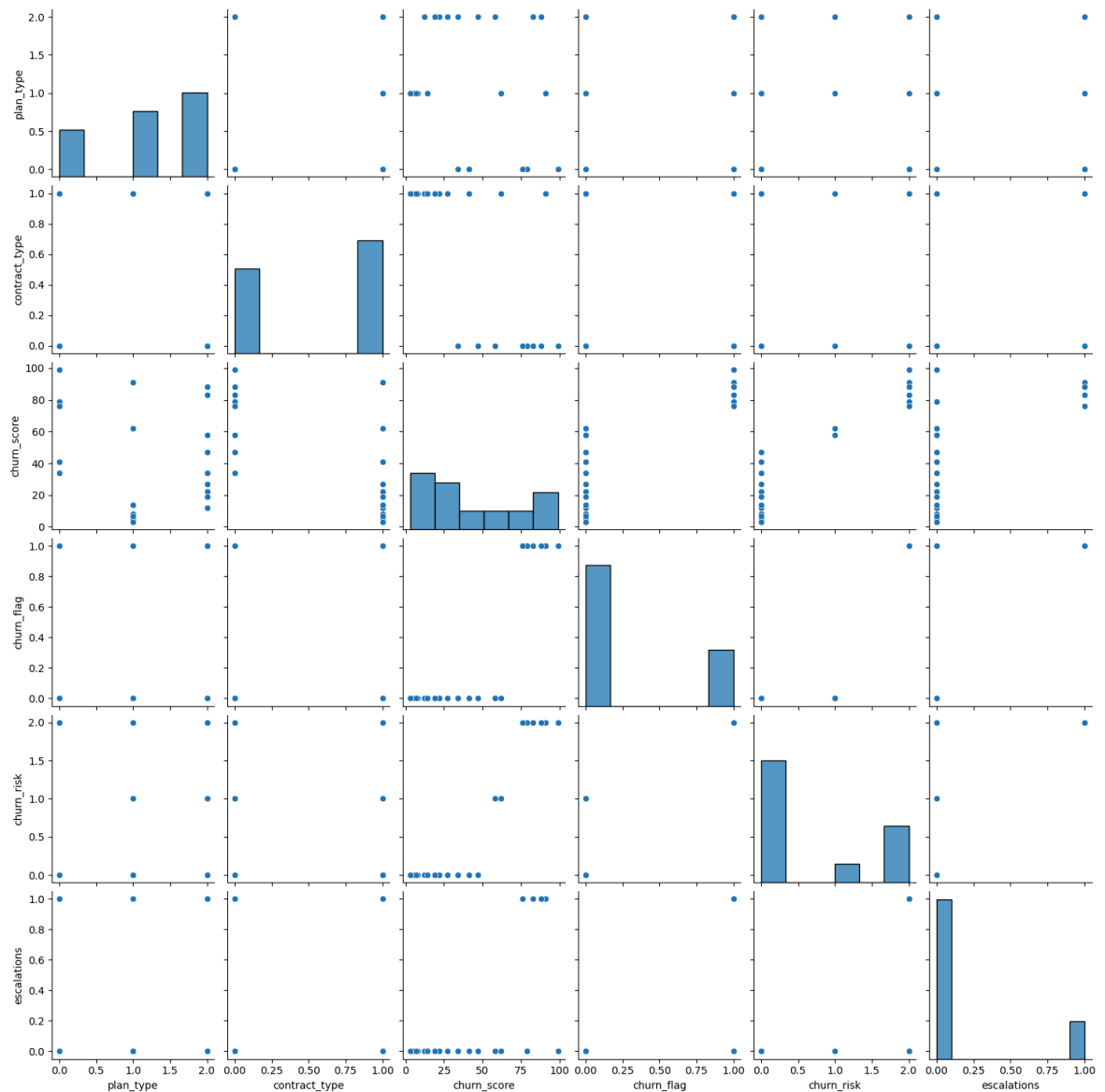
Out[72]: <Axes: >





```
In [73]: # pairplot - relashhip in a dataset  
sns.pairplot(df_encoded)
```

```
Out[73]: <seaborn.axisgrid.PairGrid at 0x1a99b343770>
```



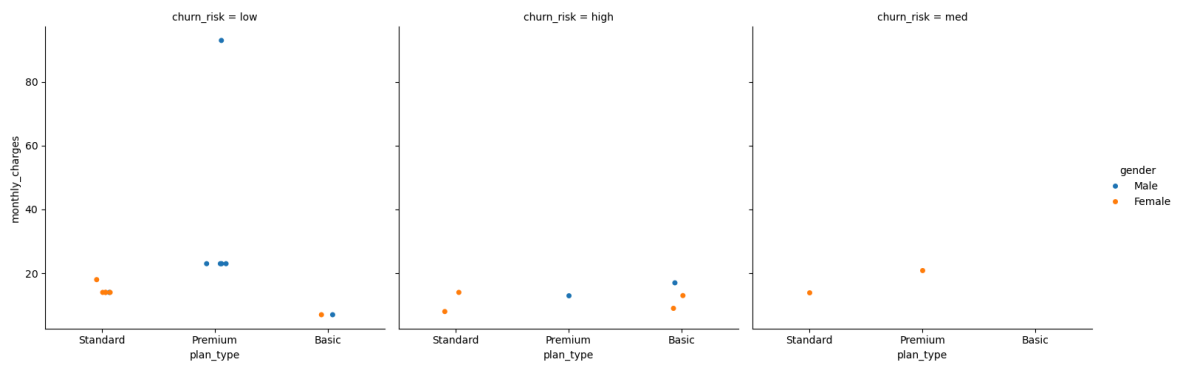
```
In [74]: df_visual.columns
```

```
Out[74]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count',
               'tenure_days', 'churn_risk', 'cancellation_month'],
              dtype='object')
```

```
In [76]: # catplt/Facegrid plot - multi-dim comparison
```

```
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

```
Out[76]: <seaborn.axisgrid.FacetGrid at 0x1a99b340d70>
```



In [ ]:

In [ ]: *# pivot Table*

```
In [78]: pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
).reset_index()
```

Out[78]:

|   | plan_type | churn_flag |
|---|-----------|------------|
| 0 | Basic     | 0.600000   |
| 1 | Premium   | 0.142857   |
| 2 | Standard  | 0.222222   |

In [80]: df\_visual.columns

Out[80]: Index(['customerid', 'subscription\_start\_date', 'subscription\_type', 'renewal\_date', 'plan\_type', 'contract\_type', 'cancellation\_date', 'cancellation\_reason', 'monthly\_charges', 'cltv', 'churn\_score', 'churn\_flag', 'customer\_name', 'country', 'state', 'gender', 'dob', 'complaint\_date', 'escalations', 'csat\_score', 'complaint\_count', 'tenure\_days', 'churn\_risk', 'cancellation\_month'], dtype='object')

```
In [82]: pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['customerid', 'monthly_charges', 'churn_flag'],
    aggfunc = {
        'customerid': 'nunique',
        'monthly_charges': 'sum',
        'churn_flag': 'mean'
    }
)
```

Out[82]:

|           | churn_flag | customerid | monthly_charges |
|-----------|------------|------------|-----------------|
| plan_type |            |            |                 |
| Basic     | 0.600000   | 5          | 52.95           |
| Premium   | 0.142857   | 7          | 218.93          |
| Standard  | 0.222222   | 9          | 123.91          |

In [ ]:

In [83]: *# Working with SQL in Python (pandas)*

In [84]: *# create db in sql*  
 conn = sqlite3.connect('test\_database.sqlite')  
  
*# table details*  
 conn.execute("CREATE TABLE users (first\_name TEXT, country TEXT, budget INTEGER)  
  
*# commit and save*  
 conn.commit()

In [87]: *# insert data*  
  
 cursor = conn.cursor()  
  
*# Write sql query to insert records in sql table*  
 cursor.execute(  
 """  
 INSERT INTO users VALUES  
 ('Tarun', 'India', '5000'),  
 ('Ayush', 'Nepal', '2400'),  
 ('Shikhar', 'Bangladesh', '2000')  
 """  
 )  
  
*# commit and save*  
 conn.commit()  
  
 print("data inserted successfully")

data inserted successfully

In [88]: *# check inserted data in table*  
 conn = sqlite3.connect('test\_database.sqlite')  
 query = """SELECT \* FROM users"""  
  
 df\_results = pd.read\_sql(query, conn)  
  
 df\_results.head()

Out[88]:

|   | first_name | country    | budget |
|---|------------|------------|--------|
| 0 | Tarun      | India      | 5000   |
| 1 | Ayush      | Nepal      | 2400   |
| 2 | Shikhar    | Bangladesh | 2000   |

```
In [89]: # aggregation
query = """
        SELECT country, sum(budget) as total_budget
        FROM users
        GROUP BY country
        """
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[89]:

|   | country    | total_budget |
|---|------------|--------------|
| 0 | Bangladesh | 2000         |
| 1 | India      | 5000         |
| 2 | Nepal      | 2400         |

```
In [90]: # always close the conn with db once the task is over
conn.close()
```

In [ ]: